LSIE-MLF

Technical Specification

Engineering Blueprint for Autonomous AI Implementation and Enterprise Deployment

Live Stream Inference Engine — Machine Learning Framework

Document Status:	**Active**
Content Version:	v3.0
Schema Version:	v3.0
Date:	2026-04-04

Contents

0 Document control and canonical terminology **3**
- 0.1 Document purpose . . . 3
- 0.2 Normative language . . . 3
- 0.3 Canonical terminology registry . . . 3
- 0.4 Version history . . . 4

1 System philosophy and operational scope **4**

2 Deterministic system data flow pipeline **5**

3 Codebase architecture and directory mapping **7**
- 3.1 Directory hierarchy . . . 7
- 3.2 Build context rules . . . 8

4 Core architecture modules **9**
- 4.A Module A — Hardware & transport . . . 9
- 4.B Module B — Ground truth ingestion . . . 11
- 4.C Module C — Orchestration & synchronization . . . 12
- 4.D Module D — Multimodal ML processing . . . 14
- 4.E Module E — Experimentation & analytics . . . 16
- 4.F Module F — Context enrichment . . . 16

5 Data governance and security **17**
- 5.1 Ephemeral vault specification . . . 17
- 5.2 Data classification . . . 18

6 Interface contracts and payload schemas **18**
- 6.1 InferenceHandoffPayload — JSON schema draft 07 . . . 19

7 Mathematical specifications **20**
- 7A AU12 facial action unit geometry . . . 21
- 7B Thompson sampling reward pipeline . . . 24

8 Deterministic LLM prompt specification **28**
- 8.1 Inference parameter constraints . . . 28
- 8.2 Expected output schema . . . 29
- 8.3 System prompt (canonical) . . . 29

9 Docker compose runtime topology **29**
- 9.1 Container topology . . . 30
- 9.2 Volume mounts . . . 30
- 9.3 GPU allocation . . . 31
- 9.4 Device passthrough . . . 31
- 9.5 Network configuration . . . 31
- 9.6 Service dependency order . . . 31

10 Dependency and runtime matrix **31**
10.1 System requirements 31
10.2 Python package pinning 32

11 Variable extraction matrix **33**

12 Error handling and recovery specification **35**
12.1 Network disconnection 35
12.2 Hardware device loss 35
12.3 Worker process crash 36
12.4 Queue overload 36

13 Autonomous implementation audit checklist **37**

0. Document control and canonical terminology

0.1 Document purpose

This document is the single authoritative specification for the Live Stream Inference Engine — Machine Learning Framework (LSIE-MLF). A developer or autonomous AI agent is required to build the complete LSIE-MLF system using only this document. No external documentation is assumed or required.

0.2 Normative language

Throughout this specification, the following terms carry precise normative meaning:

Term	Meaning
MUST / SHALL / IS REQUIRED TO	The requirement is absolute and non-negotiable.
MUST NOT / SHALL NOT	The prohibition is absolute.
IS DEFINED AS	The term has a single, unambiguous meaning as stated.

0.3 Canonical terminology registry

Every system component has exactly one canonical name. All prior synonyms are retired. Implementations MUST use only the canonical identifiers listed below.

Canonical name	Retired synonyms	Definition
API Server	api, FastAPI server, web server, ASGI server	The FastAPI application process that serves REST endpoints on port 8000. Container name: api.
ML Worker	worker, Celery node, Celery worker, GPU worker, inference worker	The Celery consumer process that executes all GPU-bound ML inference tasks. Container name: worker.
Message Broker	Redis, queue, task queue, broker	The Redis in-memory data store that brokers Celery task queues between the API Server and the ML Worker. Container name: redis.
Persistent Store	postgres, PostgreSQL, database, relational database, DB	The PostgreSQL relational database that persists analytical metrics and experiment state. Container name: postgres.
Capture Container	stream_scrcpy, scrcpy container, capture service, stream ingester	The privileged Docker container responsible for physical USB I/O and raw audio-visual stream extraction. Container name: stream_scrcpy.
IPC Pipe	FIFO, named pipe, POSIX pipe, audio pipe, kernel pipe, video pipe	POSIX Named Pipes located at `/tmp/ipc/audio_stream.raw` (audio) and `/tmp/ipc/video_stream.mkv` (video) used for zero-copy data transfer between the Capture Container and the Orchestrator Container.

Canonical name	Retired synonyms	Definition
Ephemeral Vault	24-hour vault, data vault, transient storage, secure buffer	The AES-256-GCM encrypted transient storage regime for raw media buffers, subject to mandatory 24-hour secure deletion.
InferenceHandoffPayload	handoff schema, payload schema, inference payload	The JSON Schema Draft 07 contract governing data exchange between Module C and Module D, and from Module D to Module E.
Orchestrator Container	orchestrator, segment producer, Module C worker	Dedicated Docker container running the Module C orchestration loop. Uses the same image as the ML Worker but with a different CMD (`python3.11-mservices.worker.run_orchestrator`). Produces `Celery` tasks consumed by the ML Worker.

0.4 Version history

Version	Date	Description
v1.0	2026-03-12	Initial unified specification.
v2.0	2026-03-12	Production-hardened. Applied 15-rule deterministic audit: canonical terminology, module contracts, data-flow pipeline, dependency pinning, AU12 Python fix, IPC lifecycle, temporal sync formulas, Ephemeral Vault hardening, Docker topology expansion, LLM determinism constraints, error-handling matrices, variable matrix expansion, and autonomous implementation audit.
v3.0	2026-04-01	Mathematical recipe (Thompson Sampling, continuous reward, fractional Beta-Bernoulli update), E2E gap fixes, hardening baseline, 6 spec amendments incorporated (cuDNN 8, Ubuntu 24.04, orchestrator container, scrcpy v3.3.4, audio chunk 1/30s, INT8 locked), 63 new tests, reward pipeline formalization.

1. System philosophy and operational scope

LSIE-MLF ingests raw multimodal data streams from USB-tethered mobile hardware — uncompressed PCM audio, H.264 video, and WebSocket chat metadata — and executes concurrent ML inference pipelines to produce structured analytical metrics. The system uses Thompson Sampling to adaptively optimize greeting-line interactions with live stream creators, using facial affect (AU12 smile intensity) as a dense proxy reward for the sparse macro-conversion event (creator follow).

The architecture enforces a schema-first data exchange paradigm, modular boundary constraints, and absolute dependency determinism. Every layer — from physical USB transport via ADB protocols to INT8-quantized transformer inference — MUST behave predictably. Audio arrives as uncompressed PCM, video frames are subject to lighting and head-pose variation, and chat WebSockets operate on independent network clocks. Processing this chaotic multimodal input into structured intelligence requires strict temporal synchronization and deterministic error recovery at every pipeline stage.

The system is bound by rigorous data governance protocols. Because the inference pipelines extract high-fidelity biometric data (facial landmarks) and process raw human speech, the architecture im-

plements the Ephemeral Vault policy (§5). This policy ensures compliance with data minimization principles, GDPR, and HIPAA by strictly prohibiting the persistent storage of raw, personally identifiable media. All biometric data is reduced to anonymized geometric ratios before reaching the Persistent Store.

2. Deterministic system data flow pipeline

This section defines the complete end-to-end data path through the LSIE-MLF system. Every transition specifies the transport mechanism, payload format, synchronization mechanism, and error behavior.

1. Mobile hardware → capture container

Transport	USB passthrough via `/dev/bus/usb` device mapping into the Capture Container; `scrcpy` v3.3.4 runs as two coordinated instances, with an audio process (`--no-video` plus `--record` to the audio `IPC` Pipe) on port range 27100:27199 and a video process (`--no-audio` plus `--record` to the video `IPC` Pipe) on port range 27200:27299.
Payload	Audio: raw `PCM` signed 16-bit little-endian (`s16le`), 48 kHz mono written to `/tmp/ipc/audio_stream.raw`. Video: `H.264` carried in an `MKV` streaming container written to `/tmp/ipc/video_stream.mkv`.
Sync	Android hardware clock polled via `adbshell` 'echo `$EPOCHREALTIME`' every 30 seconds; drift offset applied to timestamps.
Error behavior	If USB connection is lost, the Capture Container logs the event, halts pipe writes, and polls reconnection every 2 seconds for 60 seconds before graceful termination.

2. Capture container → orchestrator container

Transport	Audio `IPC` Pipe at `/tmp/ipc/audio_stream.raw` via the shared Docker volume ipc-share, consumed by the Orchestrator Container's `FFmpeg AudioResampler` subprocess. In parallel, the video `IPC` Pipe `/tmp/ipc/video_stream.mkv` is consumed by `PyAV` through `VideoCapture` in the same Orchestrator Container.
Payload	Continuous byte stream of `PCM s16le` audio at 48 kHz.
Sync	The audio `FIFO` is held open with `exec3<>/tmp/ipc/audio_stream.raw`, while the video `FIFO` remains unshielded so a broken `PyAV` reader forces a fresh `MKV` header on restart.
Error behavior	If the Orchestrator Container audio reader crashes, pipe writes return `EAGAIN` once the buffer fills. Overflowed bytes are silently discarded until the consumer restarts.

3. Orchestrator container internal: audio resampling

Transport	`AudioResampler` launches the `FFMPEG_RESAMPLE_CMD` subprocess inside the Orchestrator Container to convert `/tmp/ipc/audio_stream.raw` from 48 kHz mono `s16le` into 16 kHz mono `s16le` on `pipe:1`.
Payload	Input: `PCM s16le` 48 kHz. Output: `PCM s16le` 16 kHz via `pipe:1`.
Sync	`FFmpeg` executes synchronously with the audio processing thread ensuring ordered delivery.
Error behavior	If `FFmpeg` exits unexpectedly, the orchestrator detects the failure and restarts the `AudioResampler` subprocess within 1 second.

4. Module b → module c

Transport	Event dictionaries appended to the orchestrator deque buffer for inclusion in the next segment window.
Payload	Python dictionaries containing `uniqueId`, `event_type`, `timestamp_utc`, and payload fields.
Sync	Events timestamped using host UTC and drift-corrected offset before entering the deque ring buffer.
Error behavior	If the `WebSocket` disconnects, exponential backoff reconnection is attempted (1s initial delay, 30s max, 10 retries).

5. Orchestrator container → ML worker

Transport	`Celery` asynchronous task dispatch via `process_segment.delay(payload)` from the Orchestrator Container, through the `Redis` Message Broker, to the ML Worker.
Payload	`InferenceHandoffPayload` JSON object validated against the `Pydantic` schema.
Sync	The Orchestrator Container assembles fixed 30-second segments, validates the payload, and dispatches it asynchronously before inference begins in the ML Worker.
Error behavior	If schema validation fails, the segment is logged and discarded while processing continues.

6. Module d → module e

Transport	`Celery` asynchronous task dispatch through the Message Broker.
Payload	JSON payload containing `session_id`, `segment_id`, `AU12` intensity, transcription, semantic match, pitch, jitter, and shimmer.
Sync	`Redis` append-only persistence ensures tasks survive broker restarts.
Error behavior	If the broker is unreachable `Celery` retries with exponential backoff up to five attempts before writing failed payloads to `/data/processed/failed_tasks/`.

7. Module e → persistent store

Transport	`psycopg2-binary` `PostgreSQL` connection pool.
Payload	`SQLINSERT` statements with parameterized queries storing metrics as `DOUBLE PRECISION` and timestamps as `TIMESTAMPTZ`.
Sync	SERIALIZABLE isolation for experiment updates and READ COMMITTED for metric inserts.
Error behavior	If the database is unreachable the module buffers up to 1000 records in memory and retries every 5 seconds before overflowing to CSV.

3. Codebase architecture and directory mapping

The system adopts a strictly segmented monorepo architecture where individual computational domains are physically isolated within the directory structure but logically unified through a root-level Docker Compose orchestrator. Shared utilities, core mathematical libraries, and interface contracts are natively accessible to all services without private package registries.

Because the API Server runs on a single-threaded asynchronous event loop (`FastAPI/Uvicorn`) and the ML Worker executes GPU-bound inference via `Celery`, the directory structure enforces a hard physical separation between API logic and ML logic. This prevents blocking and enables independent Docker layer caching: altering a lightweight API route MUST NOT force recompilation of ML Worker images containing CUDA toolchains.

3.1 Directory hierarchy

Path	Purpose
`/docker-compose.yml`	The root orchestrator defining the multi-container topology, virtual networks, GPU device reservations, and shared `IPC` volume mounts. This file is the single entry point for all container lifecycle operations.
`/services/api/`	Houses the API Server logic. Contains the `ASGI` entry point (`Uvicorn`), RESTful routing controllers, and dependency injection modules for the Persistent Store connection pool.

Path	Purpose
`/services/worker/`	Contains the ML Worker `Celery` task consumers. Houses the orchestration logic that coordinates discrete ML pipeline steps, pulling GPU-bound inference jobs from the Message Broker.
`/services/stream_ingest/`	Dedicated to the Capture Container's raw audio-visual extraction mechanisms. Contains the shell orchestrators responsible for instantiating the `IPC` Pipe and executing the `ADB` screen mirroring and audio extraction protocols.
`/packages/ml_core/`	Houses the shared mathematical and inference utilities utilized across the stack. This includes the Python reference implementations for topological facial landmark extraction, Action Unit mathematics, and audio transcription wrappers.
`/packages/schemas/`	Centralizes `Pydantic` models and JSON Schema Draft 07 definitions ensuring strict interface contracts and data validation rules across services.
`/data/raw/`	Local host volume mount for transient staging of extracted audio-visual buffers including `IPC` pipe buffers and raw `PCM` streams. Mounted as `tmpfs` in production.
`/data/interim/`	Storage for intermediate processing states such as downsampled audio buffers and cropped video frames for localized neural network analysis.
`/data/processed/`	Destination for validated ML inference outputs and structured analytical JSON payloads prior to database ingestion.
`/requirements/`	Maintains deterministic dependency tracking separating base requirements, API dependencies, and heavy ML dependencies for optimized Docker layer caching.
`/data/sql/`	`PostgreSQL` DDL and seed data files. Mounted read-only to `docker-entrypoint-initdb.d/` for automatic schema initialization on first container startup. Contains experiment definition inserts and metrics table schemas.
`/services/worker/pipeline/`	Module C orchestration and Module E analytics pipeline. Houses the orchestrator, reward computation, and video capture pipeline modules used by the orchestrator and worker containers.

3.2 Build context rules

All Dockerfiles located in `/services` directories MUST set their build context to the monorepo root so that shared `/packages` directories are available during build. Strict `.dockerignore` rules MUST prevent unnecessary assets from entering images. The API Server container MUST exclude heavy ML dependencies, while the ML Worker containers bypass unnecessary web assets.

4. Core architecture modules

The modules defined below form the operational spine of the LSIE-MLF specification. Each module contains explicit execution details followed by a formal contract defining inputs, outputs, dependencies, side effects, and failure modes.

4.A Module A — Hardware & transport

Purpose: Extract uncompressed, zero-latency audio and video streams from tethered mobile hardware and transport them into the containerized environment without network serialization overhead.

Technology stack: scrcpy v3.3.4, dual-instance scrcpy architecture (audio on port range 27100:27199 and video on port range 27200:27299), IPC Pipe (POSIX Named Pipe) transport for separate audio and video streams, Bash file descriptor manipulation (fd 3 audio shield), Docker shared volume ipc-share

4.A.1 Execution details

Module A maps the host `/dev/bus/usb` subsystem into the Capture Container and starts two coordinated `scrcpy` v3.3.4 processes against the same Android device. One process is dedicated to audio capture and the other is dedicated to video capture, so the system no longer multiplexes both media types through a single recorder.

The dual-instance design keeps the transport path simple and restartable. Audio is written directly to `/tmp/ipc/audio_stream.raw` through `scrcpy`'s `--record` path, while video is written directly to `/tmp/ipc/video_stream.mkv`. Splitting the streams avoids cross-stream coupling and allows either `scrcpy` process to be relaunched independently when only one media path fails.

The audio pipe is the only `FIFO` held open by the shell with `exec3<>$AUDIO_PIPE`. That shield keeps the writer side alive across `scrcpy` restarts and decouples the consumer lifecycle from the producer. The video pipe is intentionally left unshielded so a `PyAV`-side failure breaks the write path, causes the video `scrcpy` process to exit, and lets the next launch emit a fresh `MKV` header.

Startup is staggered by 4 seconds to prevent concurrent `ADB` server pushes to the device. Before every capture cycle, the container clears `adbforward` and `adbreverse` tunnels, then launches the audio instance first and the video instance second. Both instances use `--no-playback` and write into the shared ipc-share volume instead of routing media through REST endpoints or `Redis` payloads.

- Audio `scrcpy` flags: `--no-video`, `--no-playback`, `--audio-codec=raw`, `--audio-buffer=30`, `--audio-dup`, `--record=$AUDIO_PIPE`, `--record-format=wav`, `--port=27100:27199`, `--tunnel-host=$HOST_IP`.
- Video `scrcpy` flags: `--no-audio`, `--no-playback`, `--video-codec=h264`, `--max-fps=30`, `--record=$VIDEO_PIPE`, `--record-format=mkv`, `--port=27200:27299`, `--tunnel-host=$HOST_IP`.

IPC pipe lifecycle specification

1. **Directory creation:** The Capture Container initializes the shared `IPC` mount by running `mkdir-p /tmp/ipc` before any `scrcpy` process is started.
2. **Pipe cleanup and creation:** `setup_pipes()` removes stale `FIFO` files with `rm-f` and recreates both `/tmp/ipc/audio_stream.raw` and `/tmp/ipc/video_stream.mkv` with `mkfifo`.

3. **Audio fd 3 shield:** The shell opens `exec3<>$AUDIO_PIPE` so the audio pipe stays open across reader or `scrcpy` restarts and the writer is decoupled from consumer timing.
4. **Video pipe left unshielded:** No extra descriptor is opened for `/tmp/ipc/video_stream.mkv`; if the `PyAV` consumer drops the pipe, the video `scrcpy` process exits and the next launch emits a fresh `MKV` header.
5. **Device wait gate:** `wait_for_device()` polls `adbdevices` every 2 seconds for up to 60 seconds and terminates the container if no Android device reports a ready device state.
6. **ADB tunnel cleanup:** Each capture loop starts by running `adbforward--remove-all` and `adbreverse--remove-all` so stale tunnels cannot collide with the audio and video port ranges.
7. **Audio stream launch:** The audio `scrcpy` instance launches first and writes directly to `/tmp/ipc/audio_stream.raw` with `--record` while binding port range 27100:27199 through `$HOST_IP`.
8. **Staggered video launch:** After a 4-second delay, the video `scrcpy` instance launches and writes `H.264` in an `MKV` streaming container directly to `/tmp/ipc/video_stream.mkv` on port range 27200:27299.
9. **Monitoring and restart:** The container waits on both `scrcpy` PIDs with `wait-n`; when either process exits, both children are cleaned up, the device is rechecked, and the dual-stream capture loop restarts.
10. **Cross-launch cleanup:** On the next container start, `setup_pipes()` again runs `rm-f` on both `FIFO` paths before recreating them, preventing stale shared-volume pipe files from surviving across launches.

Module A — Formal contract

Module A Contract

Inputs
USB-connected Android device producing raw `PCM` audio (`s16le` 48 kHz) and `H.264` video via `scrcpy`.

Outputs
Continuous `PCM` audio stream (`s16le`, 48 kHz, mono) written to `/tmp/ipc/audio_stream.raw`, and continuous `H.264` video in an `MKV` streaming container written to `/tmp/ipc/video_stream.mkv`; both are exposed as `IPC` Pipes on the shared Docker volume ipc-share.

Dependencies
`scrcpy` v3.3.4, `ADB` server via `ADB_SERVER_SOCKET=tcp:host.docker.internal:5037`, `/dev/bus/usb` device passthrough, Docker volume ipc-share mounted at `/tmp/ipc/`, environment variables `SDL_VIDEODRIVER=dummy`, `XDG_RUNTIME_DIR=/tmp`, `SCRCPY_SERVER_PATH`, and `SCRCPY_ICON_PATH`, plus dedicated `scrcpy` port ranges 27100:27199 (audio) and 27200:27299 (video).

Side effects
Creates two named pipes (`/tmp/ipc/audio_stream.raw` and `/tmp/ipc/video_stream.mkv`), opens file descriptor 3 as an audio-pipe shield, spawns persistent audio and video `scrcpy` processes, and establishes `ADB` forward/reverse tunnels for port ranges 27100:27199 and 27200:27299.

Failure modes
USB disconnect causes `wait_for_device()` to poll every 2 seconds for up to 60 seconds before the container exits. If the audio `scrcpy` process crashes, `wait-n` detects the exit, both `scrcpy` PIDs are cleaned up, and the loop restarts while fd 3 keeps `/tmp/ipc/audio_stream.raw` open. If the video path breaks, the unshielded `/tmp/ipc/video_stream.mkv` pipe lets the video `scrcpy` process fail so the next launch writes a fresh `MKV` header for `PyAV`. Each capture cycle removes adb forward/reverse tunnels to prevent port collisions, and if the downstream consumer lags, pipe writes may return `EAGAIN` so overflowed data is silently discarded per §12.

4.B Module B — Ground truth ingestion

Purpose: Ingest real-time contextual metadata, chat streams, and user interactions to create ground-truth datasets.

Technology stack: Python 3.11.x, TikTokLive connector, EulerStream signature API

4.B.1 Execution details

The system continuously ingests live chat and interaction events from the platform using the `TikTokLive` Python connector.

Connection requires ByteDance anti-crawler signatures including `XBogus` and `msToken`. Signature generation is delegated to the `EulerStream` signature server.

Once connected the ingestion module enforces the `Action_Combo` constraint where multiple events must coincide temporally to form a valid ground truth signal.

Incoming protobuf messages are parsed into Python dictionaries and high-value composite events are staged for later synchronization with inference outputs.

> **Integration Status**
> Module B is fully implemented and tested in isolation but is not wired into the orchestrator loop. Integration is deferred pending availability of the EulerStream third-party signature API required for TikTok WebSocket authentication. Wiring Module B into the orchestrator is scoped as a feature update task, not a hardening task.

Module B — Formal contract

Module B Contract

Inputs
Authenticated TikTok Webcast `WebSocket` event stream using `EulerStream`-signed URL parameters.

Outputs
Parsed dictionaries representing high-value events containing `uniqueId`, `event_type`, `timestamp_utc`, `gift_value`, and `combo_events`.

Dependencies
`TikTokLive` library, `EulerStream` signature API, network access to TikTok Webcast servers.

Side effects
Maintains persistent `WebSocket` connection and refreshes signature tokens during reconnects.

Failure modes
`WebSocket` disconnection triggers exponential backoff reconnect. `EulerStream` outage results in degraded mode. Malformed protobuf events are logged and discarded.

4.C Module C — Orchestration & synchronization

Purpose: Module C runs as a dedicated Orchestrator Container built from the worker image but started with `python3.11-mservices.worker.run_orchestrator`. It coordinates real-time stream processing by polling `ADB` for temporal drift correction, resampling 48 kHz `PCM` to 16 kHz through a persistent `FFmpeg` subprocess, processing video frames for `AU12` facial telemetry, assembling fixed 30-second segments, and dispatching `InferenceHandoffPayload` tasks to the `Celery` worker. In this split architecture the orchestrator is the producer of inference tasks and the ML Worker is the consumer.

Technology stack: asyncio event loop, FFmpeg persistent resampling subprocess, PyAV-backed video pipe consumer, MediaPipe Face Mesh via packages/ml_core, ADB epoch polling, Celery task dispatch, Redis broker, psycopg2/PostgreSQL session registration and experiment lookup, collections.deque buffering

4.C.1 Temporal drift correction

`DriftCorrector` polls the Android epoch every 30 seconds by invoking `ADB` and computes the drift offset as host UTC minus Android epoch before segment timestamps are emitted.

Each corrected timestamp is produced by adding the cached drift offset to the original timestamp, and the orchestrator warns when absolute drift exceeds 150 milliseconds.

After three consecutive `ADB` poll failures the corrector freezes the last cached offset and returns that value while frozen. If the frozen state lasts 300 seconds, the offset is reset to zero so timestamp correction can resume from a neutral baseline on the next successful poll.

4.C.2 Video telemetry and segment assembly

The Orchestrator Container keeps a long-lived video capture thread on `/tmp/ipc/video_stream.mkv`, samples the latest frame inside the asyncio loop, and lazy-loads `MediaPipe FaceMesh` together with the `AU12` normalizer on first use.

Before stimulus injection, incoming frames only accumulate the `B_neutral` calibration baseline. After the greeting trigger is recorded, the orchestrator stores a drift-corrected stimulus timestamp and appends per-frame `AU12` observations `\{timestamp_s,intensity\}` to `_au12_series` for downstream reward computation.

In parallel, audio bytes accumulate until a full 30-second window is available. The orchestrator then drains the event deque, assembles an `InferenceHandoffPayload`, attaches `_active_arm`, `_experiment_id`, `_expected_greeting`, `_stimulus_time`, `_au12_series`, and `_x_max`, base64-encodes binary fields for `Celery` JSON transport, and dispatches the segment to Module D.

4.C.3 Audio resampling

`AudioResampler` launches a persistent `FFmpeg` subprocess inside the Orchestrator Container to convert the raw 48 kHz mono `IPC` stream at `/tmp/ipc/audio_stream.raw` into 16 kHz mono `s16le PCM` emitted on `pipe:1`.

The run loop sets `bytes_per_second` = 32000 and `chunk_size` = `int(bytes_per_second/30)`, so audio is consumed in 1/30-second reads of 1,066 bytes per iteration (approximately 1,067 bytes) to stay aligned with 30 FPS video instead of using 1-second chunks.

If the `FFmpeg` subprocess is not running, `read_chunk()` waits 1 second and starts a fresh resampler. `EOF` or pipe read errors stop the old process so the next iteration can recover automatically.

Drift polling specification

Parameter	Value
Polling interval	30 seconds (DRIFT_POLL_INTERVAL)
ADB command	adb shell 'echo $EPOCHREALTIME'
Return format	Decimal epoch string with microsecond precision parsed as float
Maximum tolerated drift	150 milliseconds (MAX_TOLERATED_DRIFT_MS)
Drift formula	drift_offset = host_utc - android_epoch
Timestamp correction	corrected_ts = original_ts + drift_offset
Freeze threshold	3 consecutive ADB failures (DRIFT_FREEZE_AFTER_FAILURES)
Reset timeout	300 seconds / 5 minutes of frozen state (DRIFT_RESET_TIMEOUT)

FFmpeg resampling command

```
ffmpeg -f s16le -ar 48000 -ac 1 -i /tmp/ipc/audio_stream.raw -ar 16000 -f s16le -ac 1 pipe:1
```

Module C — Formal contract

Module C Contract

Inputs
Raw `PCM` audio from `/tmp/ipc/audio_stream.raw`, `H.264` video frames decoded from `/tmp/ipc/video_stream.mkv` by the `PyAV`-backed video capture thread, ground-truth event dictionaries drained from the in-memory deque, Android epoch timestamps returned by `ADB` drift polls, and operator stimulus trigger events used to mark greeting injection time.

Outputs
16 kHz resampled mono `PCM`, drift-corrected segment timestamps, `InferenceHandoffPayload` segments dispatched as `Celery` tasks to the ML Worker, per-frame `AU12` telemetry in `_au12_series`, stimulus timestamps in `_stimulus_time`, and experiment attribution fields `_active_arm`, `_experiment_id`, `_expected_greeting`, and `_x_max`.

Dependencies
`FFmpeg` persistent subprocess, `PyAV`-backed video capture, `MediaPipe FaceMesh` and `AU12` normalization via `packages/ml_core`, `ADB` drift polling, `Celery` task dispatch, `Redis` broker, `PostgreSQL` access for session registration and Thompson Sampling arm selection, shared Docker volume ipc-share mounted at `/tmp/ipc/`, and data-raw, data-interim, and data-processed volumes.

Side effects
Spawns a persistent `FFmpeg` subprocess, starts and revives the video capture thread, registers the session in `PostgreSQL`, selects a Thompson Sampling arm and expected greeting, accumulates `AU12` calibration and telemetry state in memory, and dispatches assembled segments to the `Celery` worker.

Failure modes
`ADB` poll failures freeze the last drift offset after 3 consecutive errors and reset it to zero after 300 seconds of frozen state. Missing or crashed `FFmpeg` subprocesses wait 1 second before automatic restart, while `EOF` and read errors stop the old process so the next iteration can recover. Video capture thread death triggers a best-effort stop/recreate/start cycle in the run loop after pipe crashes, `AU12` frame processing failures are debug-logged and skipped, event buffer overflow is bounded by `deque(maxlen=10000)` eviction, and `Celery` dispatch failures are logged while the loop continues.

4.D Module D — Multimodal ML processing

Purpose: Perform speech transcription, facial landmark inference, acoustic analysis, and semantic evaluation.

Technology stack: faster-whisper v1.2.1, mediapipe v0.10.9, praat-parselmouth v0.4.4, spacy v3.7.x

4.D.1 Speech transcription

Speech transcription is executed by `TranscriptionEngine`, which wraps `faster-whisper` with the `CTranslate2` backend and fixes the class constant `_COMPUTE_TYPE` to "int8". The worker does not expose `compute_type` as a deployment-time option.

For each segment, the `Celery` task writes the 16 kHz `s16le PCM` bytes to a temporary .raw file and then runs `FFmpeg` to wrap that raw audio in a `WAV` container before calling `engine.transcribe()`. This header-wrapping step is required so `faster-whisper` receives a properly described audio container instead of headerless `PCM`.

GPU execution targets CUDA 12 with `cuDNN` 8, matching the worker image. `cuDNN` 8 is required to preserve Pascal SM 6.1 dp4a compatibility for INT8 inference on the target hardware (GTX 1080 Ti).

4.D.2 Computer vision — face mesh

Video frames are processed using `MediaPipe FaceMesh` generating a 478-vertex 3D landmark mesh.

The model uses Procrustes Analysis to construct a normalized metric face space.

4.D.3 Acoustic analysis

The Praat engine accessed through parselmouth extracts pitch, jitter, and shimmer metrics from speech audio.

4.D.4 Text pre-processing

`spaCy` performs tokenization, normalization, and linguistic preprocessing prior to LLM semantic evaluation.

Module D — Formal contract

Module D Contract

Inputs
16 kHz audio stream, raw video frames, and `InferenceHandoffPayload` segments.

Outputs
Speech transcripts, 478-vertex facial landmark arrays, `AU12` intensity scores, acoustic metrics, semantic evaluation JSON results, and `RewardResult` payloads produced by invoking `compute_reward()` (§7B) with the segment's `_au12_series`, `_stimulus_time`, and semantic evaluation `is_match` gate.

Dependencies
`faster-whisper` 1.2.1 with `CTranslate2`, `MediaPipe` 0.10.9, `praat-parselmouth` 0.4.4, `spaCy` 3.7.*, `OpenAI` Python SDK for Azure `OpenAI` calls, `FFmpeg` for `WAV` container wrapping, and CUDA 12.2.2 runtime with `cuDNN` 8.

Side effects
Loads ML models into GPU memory and performs external HTTPS calls to Azure `OpenAI`.

Failure modes
GPU OOM triggers worker restart. Whisper load failure aborts startup. Missing face returns null facial metrics. LLM timeout retries once before recording null.

4.E Module E — Experimentation & analytics

Purpose: Aggregate inference metrics, persist results to the database, and run adaptive experimentation logic.

Technology stack: psycopg2-binary v2.9.x, pandas v2.2.x, SciPy, Streamlit

4.E.1 Execution details

High-frequency ML outputs such as `AU12` arrays, pitch trajectories, and semantic scores are aggregated into structured time-series datasets.

pandas performs high-performance data manipulation while `SciPy` provides statistical analysis.

`compute_reward()` (§7B) is the entry point for the reward pipeline. It receives per-segment `_au12_series`, `_stimulus_time`, and the semantic evaluation `is_match` boolean from Module D, computes the gated reward `r_t=p90_intensity×semantic_gate`, and returns a `RewardResult` dataclass. The `Celery` task in Module D invokes this function and persists the result; Module E owns the posterior update logic that consumes the reward.

Thompson Sampling is used to dynamically evaluate greeting rules and behavioral prompts.

Operational metrics and experiment state are visualized through a `Streamlit` dashboard.

Module E — Formal contract

Module E Contract

Inputs
Inference payloads from Module D delivered through the `Celery` task queue.

Outputs
Time-series records stored in the Persistent Store, Thompson Sampling experiment assignments and posterior updates (`α+=r_t`, `β+=1-r_t`), `RewardResult` dataclass produced by `compute_reward()` (§7B), and `Streamlit` dashboards.

Dependencies
`psycopg2-binary`, pandas, `SciPy`, `Streamlit`, `Redis` Message Broker, `PostgreSQL` database.

Side effects
Writes metrics and reward results to database tables, updates Thompson Sampling Beta posterior parameters per encounter, and runs a `Streamlit` UI service.

Failure modes
Database outage buffers 1000 records before disk overflow. `Celery` task deserialization errors are logged and discarded.

4.F Module F — Context enrichment

Purpose: Perform asynchronous scraping of external web sources to enrich contextual information about the stream.

Technology stack: Celery v5.4.x, Message Broker, patchright v1.58.2

4.F.1 Execution details

Context enrichment tasks run asynchronously outside the main inference loops.

`Celery` workers execute scraping jobs that gather external metadata about creators or trending topics.

`patchright` is used instead of standard Playwright to bypass bot-detection mechanisms by patching `Chromium` automation fingerprints.

Module F — Formal contract

Module F Contract

Inputs
`Celery` tasks containing target URL, `scrape_type`, and `timeout_seconds` parameters.

Outputs
JSON objects containing scraped metadata including `source_url`, `scraped_at_utc` timestamp, and extracted data dictionary.

Dependencies
`Celery`, `Redis` Message Broker, `patchright` browser automation toolkit, `Chromium` runtime.

Side effects
Launches ephemeral headless browser instances and performs outbound HTTP requests.

Failure modes
CAPTCHA or HTTP blocking returns empty result. Browser crash triggers retry. Timeout terminates browser. Broker outage triggers `Celery` reconnection.

5. Data governance and security

The LSIE-MLF processes highly sensitive multimodal data including raw speech audio and dense 3D facial topology maps. Operation of such a system requires strict regulatory compliance, strong encryption practices, and strict data minimization policies to ensure that personally identifiable biometric information never persists beyond the transient processing pipeline.

5.1 Ephemeral vault specification

Parameter	Specification
Encryption algorithm	`AES-256-GCM` (authenticated encryption with associated data)
Encryption library	`PyCryptodome` (pycryptodome >= 3.20.0)
Key length	256 bits

Parameter	Specification
IV/Nonce length	96 bits (12 bytes) generated using `os.urandom(12)`
Key generation	`os.urandom(32)` generating a unique key per debug session
Key storage location	Process memory only. Keys are never written to disk, environment variables, or configuration files.
Key lifecycle	Generated at session start and destroyed when the container terminates. No persistence mechanism exists.
Secure deletion method	`shred-vfz-n3` executed by internal cron every 24 hours on `/data/raw/` and `/data/interim/`
Retention period	Maximum 24 hours for any raw media buffer touching physical disk

5.2 Data classification

Classification	Description	Storage	Retention
Transient Data	Raw `PCM` audio buffers and uncompressed video frames existing only in volatile memory during processing.	`/tmp/ipc/(tmpfs)andkernelpipebuffers`	Duration of active processing only with no disk persistence.
Debug Storage	Encrypted diagnostic media buffers written to disk only when algorithmic anomalies occur.	`/data/raw/and/data/interim/`	Maximum 24 hours followed by secure deletion.
Permanent Analytical Storage	Anonymized analytical metrics including `AU12` scores, acoustic metrics, and semantic evaluation results.	`PersistentStore(PostgreSQL)`	Indefinite subject to database backup policies.

The system MUST never store raw facial images, voiceprints, or other personally identifiable biometric media in the Persistent Store. MediaPipe landmarks are reduced to normalized geometric ratios such as AU12 intensity values before persistence, ensuring biometric anonymization.

6. Interface contracts and payload schemas

In distributed multimodal machine learning pipelines, loosely typed interfaces between processing modules can cause catastrophic runtime failures. Incorrect serialization of numeric arrays or omission of required fields can break downstream indexing and database ingestion.

To guarantee deterministic communication between modules, LSIE-MLF enforces strict interface contracts defined using JSON Schema Draft 07 and validated using `Pydantic` models within the Python runtime.

The schema below defines both the base media transport contract (required fields) and the v3 experiment attribution fields (prefixed with underscore) that the Orchestrator Container attaches before `Celery` dispatch. The experiment fields carry Thompson Sampling arm selection, stimulus timing, per-frame `AU12` telemetry, and calibration state needed by the reward pipeline (§7B).

6.1 InferenceHandoffPayload — JSON schema draft 07

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "InferenceHandoffPayload",
  "description": "Standardized schema for multimodal ML pipeline handoff. Base contract defines media transport
  ↪  fields; v3 experiment attribution fields support Thompson Sampling reward computation.",
  "type": "object",
  "required": [
    "session_id",
    "timestamp_utc",
    "media_source",
    "segments"
  ],
  "properties": {
    "session_id": {
      "type": "string",
      "pattern": "^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$",
      "description": "UUID v4 representing the continuous live stream session."
    },
    "timestamp_utc": {
      "type": "string",
      "format": "date-time",
      "description": "RFC 3339 UTC timestamp of inference event completion."
    },
    "media_source": {
      "type": "object",
      "required": [
        "stream_url",
        "codec",
        "resolution"
      ],
      "properties": {
        "stream_url": {
          "type": "string",
          "format": "uri"
        },
        "codec": {
          "type": "string",
          "enum": [
            "h264",
            "h265",
            "raw"
          ]
        },
        "resolution": {
          "type": "array",
          "items": {
            "type": "integer",
            "minimum": 1
          },
          "minItems": 2,
          "maxItems": 2,
          "description": "[width, height] in pixels."
        }
      }
    },
    "segments": {
```

```
      "type": "array",
      "description": "Array of analyzed video segments."
    },
    "_active_arm": {
      "type": "string",
      "description": "Thompson Sampling arm identifier selected for this segment."
    },
    "_experiment_id": {
      "type": "integer",
      "description": "Persistent Store experiment row ID for this session."
    },
    "_expected_greeting": {
      "type": "string",
      "description": "Greeting rule text assigned to the active arm for semantic evaluation."
    },
    "_stimulus_time": {
      "type": [
        "number",
        "null"
      ],
      "description": "Drift-corrected UTC epoch seconds of stimulus injection, or null if no stimulus occurred
      ↪  in this segment."
    },
    "_au12_series": {
      "type": "array",
      "items": {
        "type": "object",
        "required": [
          "timestamp_s",
          "intensity"
        ],
        "properties": {
          "timestamp_s": {
            "type": "number"
          },
          "intensity": {
            "type": "number",
            "minimum": 0.0,
            "maximum": 1.0
          }
        }
      },
      "description": "Per-frame AU12 bounded intensity observations from the orchestrator."
    },
    "_x_max": {
      "type": [
        "number",
        "null"
      ],
      "description": "Per-subject maximum raw D_mouth/IOD ratio observed during calibration, used for optional
      ↪  range normalization. Null if calibration is incomplete."
    }
  }
}
```

7. Mathematical specifications

7A AU12 facial action unit geometry

The quantitative measurement of human facial affect is governed by the Facial Action Coding System (FACS). Within this taxonomy Action Unit 12 (AU12), the Lip Corner Puller, corresponds to contraction of the zygomaticus major muscle drawing the mouth corners upward and outward.

Module D computes AU12 intensity from the 478 three-dimensional landmarks generated by the MediaPipe Face Mesh model. Because head distance and pose distort raw pixel distances, geometric normalization is performed using the interocular distance (IOD) as a scale-invariant reference computed in full 3D Euclidean space.

7A.1 Variable dictionary

Symbol	Code name	Definition
L_i	`landmarks[i]`	3D vector (x_i, y_i, z_i) of MediaPipe landmark i.
L_61	`landmarks[61]`	Left lip corner landmark.
L_291	`landmarks[291]`	Right lip corner landmark.
L_33	`landmarks[33]`	Left eye outer corner.
L_133	`landmarks[133]`	Left eye inner corner.
L_362	`landmarks[362]`	Right eye inner corner.
L_263	`landmarks[263]`	Right eye outer corner.
E_L	`e_l`	Geometric center of left eye midpoint(L_33,L_-133).
E_R	`e_r`	Geometric center of right eye midpoint(L_-362,L_263).
IOD	`iod`	3D Euclidean distance between E_L and E_R.
D_mouth	`d_mouth`	3D Euclidean distance between lip corners.
$B_neutral$	`self.b_neutral`	Neutral baseline average of D_mouth/IOD.
$I_AU12(t)$	`i_au12`	Normalized AU12 intensity at time t.
$alpha$	`self.alpha`	Empirical linear projection multiplier. Default 6.0 (v3.0 FACS-anchored calibration: 95th percentile Duchenne deviation approximately 0.15 maps to 0.90 via alpha_scale = 0.90 / 0.15 = 6.0).
$Score_AU12(t)$	`score_au12`	Final AU12 FACS score in range 0–5.
$Score_{AU12,bounded}(t)$	`compute_bounded_intensity()`	AU12 intensity bounded to [0.0, 1.0] via tanh soft-saturation. Output method for the fractional Beta-Bernoulli Thompson Sampling reward pipeline.

7A.2 Step 1: Topological landmark extraction

The anatomical points required for AU12 computation are extracted from the MediaPipe landmark array.

- Mouth corners: L_61 (left) and L_291 (right)
- Left eye bounds: L_33 outer and L_133 inner
- Right eye bounds: L_362 inner and L_263 outer

7A.3 Step 2: Derivation of interocular distance (IOD)

Eye centers are computed as midpoints between inner and outer eye landmarks and used to calculate the normalization scalar.

$$E_L = ((x_{33}+x_{133})/2, (y_{33}+y_{133})/2, (z_{33}+z_{133})/2); \quad E_R = ((x_{362}+x_{263})/2, (y_{362}+y_{263})/2, (z_{362}+z_{263})/2)$$

$$IOD = \sqrt{(E_R.x - E_L.x)^2 + (E_R.y - E_L.y)^2 + (E_R.z - E_L.z)^2}$$

7A.4 Step 3: AU12 distance and baseline calibration

The raw AU12 distance equals the Euclidean distance between lip corners.

A neutral baseline B_neutral is computed from the rolling average of D_mouth/IOD during the calibration phase.

The bounded output maps [0, +infinity) to [0, 1) via tanh soft-saturation, providing a continuous gradient at the boundaries suitable for fractional Beta-Bernoulli posterior updates. With alpha_-scale=6.0, a Duchenne smile (deviation approximately 0.15) maps to tanh(0.9) approximately 0.72, and laughter (deviation approximately 0.20) maps to tanh(1.2) approximately 0.83.

$$D_{mouth} = \sqrt{(x_{291} - x_{61})^2 + (y_{291} - y_{61})^2 + (z_{291} - z_{61})^2}$$

$$I_{AU12}(t) = (D_{mouth}(t)/IOD(t)) - B_{neutral}$$

$$Score_{AU12}(t) = max(0, \alpha \cdot I_{AU12}(t))$$

$$Score_{AU12,bounded}(t) = \tanh\big(\alpha \cdot \max(0,\ I_{AU12})\big)$$

7A.5 AU12Normalizer — reference implementation

```
from __future__ import annotations
import math
from typing import Any
import numpy as np
import numpy.typing as npt

EPSILON: float = 1e-6
DEFAULT_ALPHA_SCALE: float = 6.0  # §7A: 0.90 / 0.15 = 6.0
```

```python
class AU12Normalizer:
    def __init__(self, alpha: float = DEFAULT_ALPHA_SCALE) -> None:
        self.alpha: float = alpha
        self.b_neutral: float | None = None
        self.calibration_buffer: list[float] = []

    def _extract_geometry(
        self, landmarks: npt.NDArray[np.floating[Any]]
    ) -> tuple[float, float] | None:
        left_eye_outer = landmarks[33]
        left_eye_inner = landmarks[133]
        right_eye_inner = landmarks[362]
        right_eye_outer = landmarks[263]
        left_lip = landmarks[61]
        right_lip = landmarks[291]

        e_l = (left_eye_outer + left_eye_inner) / 2.0
        e_r = (right_eye_inner + right_eye_outer) / 2.0
        iod: float = float(np.linalg.norm(e_r - e_l))
        if iod < EPSILON:
            return None
        d_mouth: float = float(np.linalg.norm(right_lip - left_lip))
        return (iod, d_mouth)

    def _update_calibration(self, ratio: float) -> None:
        self.calibration_buffer.append(ratio)
        self.b_neutral = float(np.mean(self.calibration_buffer))

    def compute_raw_ratio(
        self, landmarks: npt.NDArray[np.floating[Any]]
    ) -> float | None:
        geom = self._extract_geometry(landmarks)
        if geom is None:
            return None
        iod, d_mouth = geom
        return d_mouth / iod

    def compute_intensity(
        self, landmarks: npt.NDArray[np.floating[Any]], is_calibrating: bool = False
    ) -> float:
        geom = self._extract_geometry(landmarks)
        if geom is None:
            return 0.0
        iod, d_mouth = geom
        ratio = d_mouth / iod
        if is_calibrating:
            self._update_calibration(ratio)
            return 0.0
        if self.b_neutral is None:
            raise ValueError("Baseline not calibrated")
        score = self.alpha * (ratio - self.b_neutral)
        return float(min(max(score, 0.0), 5.0))

    def compute_bounded_intensity(
        self, landmarks: npt.NDArray[np.floating[Any]], is_calibrating: bool = False
    ) -> float:
```

```
        geom = self._extract_geometry(landmarks)
        if geom is None:
            return 0.0
        iod, d_mouth = geom
        ratio = d_mouth / iod
        if is_calibrating:
            self._update_calibration(ratio)
            return 0.0
        if self.b_neutral is None:
            raise ValueError("Baseline not calibrated")
        deviation = max(0.0, ratio - self.b_neutral)
        return float(math.tanh(self.alpha * deviation))
```

Version Corrections

v2.0: Landmark indexing corrected to use `landmarks[i]`; epsilon guard added for `IOD` → 0; output hard-clamped to 5.0; full type annotations added.
v3.0: `alpha_scale` changed from 5.0 to 6.0 (FACS-anchored: 95th percentile Duchenne deviation ≈ 0.15 maps to 0.90). Added `compute_bounded_intensity()` via tanh $[0, 1]$ for fractional Beta-Bernoulli Thompson Sampling reward pipeline. Added `compute_raw_ratio()` for per-subject range normalization. Shared geometry extraction refactored into `_extract_geometry()`.

7B Thompson sampling reward pipeline

The LSIE-MLF uses Thompson Sampling to adaptively select greeting strategies that maximize creator engagement. The reward signal is derived from facial affect: AU12 smile intensity (§7A) serves as a dense proxy for the sparse macro-conversion event (creator follow). This section formalizes the reward computation pipeline that transforms per-frame AU12 observations into a scalar reward signal suitable for fractional Beta-Bernoulli posterior updates.

The core design decision is a binary semantic validity gate rather than continuous confidence weighting. The gate ensures that only encounters where the greeting was actually delivered (confirmed by LLM semantic evaluation, §8) contribute to the posterior. This prevents accumulated false positive contamination: if the greeting was not spoken, the smile response — however strong — is not attributable to the experimental arm and MUST NOT update the posterior.

7B.1 Variable dictionary

Symbol	Code name	Definition
t_{stim}	`stimulus_time_s`	Drift-corrected UTC epoch timestamp of greeting stimulus injection.
$W_{measure}$	`extract_stimulus_window()`	Stimulus-locked measurement window [`t_stim` + 0.5s, `t_stim` + 5.0s].
$W_{baseline}$	`extract_stimulus_window(baseline)`	Pre-stimulus baseline window [`t_stim` - 5.0s, `t_stim` - 2.0s].
P_{90}	`p90_intensity`	90th percentile of bounded `AU12` intensities within `W_measure`.
G_t	`semantic_gate`	Binary semantic validity gate: 1 if `is_match` is True, 0 otherwise.
r_t	`gated_reward`	Gated reward signal: `r_t` = `P_90` × `G_t`. Range [0.0, 1.0].
α	`posterior_alpha`	Beta distribution alpha parameter (success count). Prior: 1.0.
β	`posterior_beta`	Beta distribution beta parameter (failure count). Prior: 1.0.
x_{max}	`_x_max`	Per-subject maximum raw `D_mouth/IOD` ratio from calibration, used for optional range normalization.
$B_{neutral,pre}$	`baseline_b_neutral`	Mean `AU12` bounded intensity in `W_baseline`, or null when no pre-stimulus frames exist.

7B.2 Step 1: Signal source and bounded output

The reward pipeline consumes per-frame `AU12` bounded intensities produced by `compute_bounded_intensity()` (§7A, Step 3). Each observation is a `\{timestamp_s,intensity\}` pair where intensity is in [0.0, 1.0] via tanh soft-saturation. The bounded output is essential: the fractional Beta-Bernoulli update requires rewards in [0, 1], and the tanh mapping provides a continuous gradient at the boundaries that prevents posterior corruption.

7B.3 Step 2: Stimulus window extraction

The measurement window is defined as [`t_stim` + 0.5s, `t_stim` + 5.0s]. The 0.5-second onset delay accounts for the physiological latency between hearing the greeting and producing a facial response. The 5.0-second end bound captures the full smile trajectory including onset, apex, and initial decay.

A pre-stimulus baseline window [`t_stim` - 5.0s, `t_stim` - 2.0s] is computed for diagnostic purposes. The 2.0-second guard prevents contamination from anticipatory expressions triggered by the operator's preparation to speak.

$$W_{measure} = \{(t_i, s_i) \in \text{au12_series} \mid t_{stim} + 0.5 \leq t_i \leq t_{stim} + 5.0\}$$

$$W_{baseline} = \{(t_i, s_i) \in \text{au12_series} \mid t_{stim} - 5.0 \leq t_i \leq t_{stim} - 2.0\}$$

7B.4 Step 3: P90 aggregation and gated reward

The 90th percentile is used instead of the mean or maximum to reduce sensitivity to single-frame noise spikes while still capturing the peak smile response. If the measurement window contains zero frames, P90 defaults to 0.0.

The semantic gate `G_t` is a binary multiplier derived from the `is_match` boolean in the LLM semantic evaluation output (§8). It is intentionally binary rather than using `confidence_score` as a continuous weight. Continuous gating would allow partially confident matches to drip reward into the posterior, accumulating false positive contamination over many encounters. Binary gating enforces a clean decision boundary: either the greeting was delivered and the smile is attributable, or it was not.

$$P_{90} = \text{percentile}_{90}\big(\{s_i \mid (t_i, s_i) \in W_{measure}\}\big)$$

$$G_t = \begin{cases} 1 & \text{if is_match} = \text{True} \\ 0 & \text{otherwise} \end{cases}$$

$$r_t = P_{90} \times G_t$$

7B.5 Step 4: Fractional beta-bernoulli posterior update

Each arm maintains a Beta posterior initialized with a uniform prior $\text{Beta}(1, 1)$. After each encounter, the posterior is updated with the fractional reward `r_t` $\in [0, 1]$. Unlike standard Bernoulli updates that require binary 0/1 outcomes, the fractional update preserves the continuous gradient of the bounded AU12 signal.

Arm selection uses Thompson Sampling: for each arm, a value is sampled from its current Beta posterior, and the arm with the highest sampled value is selected for the next encounter.

$$\alpha \leftarrow \alpha + r_t, \quad \beta \leftarrow \beta + (1 - r_t)$$

$$a^* = \arg\max_a \theta_a, \quad \theta_a \sim \text{Beta}(\alpha_a, \beta_a)$$

7B.6 Reward pipeline and Thompson Sampling — reference implementation

```python
from __future__ import annotations
import random
from dataclasses import dataclass
from typing import Optional
import numpy as np

STIMULUS_WINDOW_START: float = 0.5   # seconds after stimulus
STIMULUS_WINDOW_END: float = 5.0     # seconds after stimulus
BASELINE_WINDOW_START: float = -5.0  # seconds before stimulus
BASELINE_WINDOW_END: float = -2.0    # seconds before stimulus
DEFAULT_PRIOR_ALPHA: float = 1.0
DEFAULT_PRIOR_BETA: float = 1.0


@dataclass
class RewardResult:
```

```python
    gated_reward: float
    p90_intensity: float
    semantic_gate: int
    n_frames_in_window: int
    baseline_b_neutral: Optional[float]


def extract_stimulus_window(
    au12_series: list[dict], stimulus_time_s: float,
    start_offset: float = STIMULUS_WINDOW_START,
    end_offset: float = STIMULUS_WINDOW_END,
) -> list[dict]:
    t_start = stimulus_time_s + start_offset
    t_end = stimulus_time_s + end_offset
    return [o for o in au12_series if t_start ≤ o["timestamp_s"] ≤ t_end]


def compute_reward(
    au12_series: list[dict], stimulus_time_s: float,
    is_match: bool, x_max: Optional[float] = None,
) -> RewardResult:
    window = extract_stimulus_window(au12_series, stimulus_time_s)
    n_frames = len(window)

    if n_frames == 0:
        p90 = 0.0
    else:
        p90 = float(np.percentile([o["intensity"] for o in window], 90))

    if x_max is not None and x_max > 0:
        p90 = min(p90 / x_max, 1.0)

    gate = 1 if is_match else 0
    gated = p90 * gate

    baseline_obs = extract_stimulus_window(
        au12_series, stimulus_time_s,
        start_offset=BASELINE_WINDOW_START, end_offset=BASELINE_WINDOW_END,
    )
    baseline = (
        float(np.mean([o["intensity"] for o in baseline_obs]))
        if baseline_obs else None
    )

    return RewardResult(
        gated_reward=gated, p90_intensity=p90, semantic_gate=gate,
        n_frames_in_window=n_frames, baseline_b_neutral=baseline,
    )


def thompson_sample_arm(arms: dict[str, tuple[float, float]]) -> str:
    samples = {
        arm_id: random.betavariate(alpha, beta)
        for arm_id, (alpha, beta) in arms.items()
    }
    return max(samples, key=samples.get)
```

```python
def update_posterior(
    alpha: float, beta: float, reward: float
) -> tuple[float, float]:
    return alpha + reward, beta + (1.0 - reward)
```

8. Deterministic LLM prompt specification

Module D must determine whether a spoken utterance satisfies a predefined semantic rule such as a required greeting phrase. Traditional string-matching algorithms like Levenshtein distance or BLEU fail because human speech contains fillers, paraphrasing, and transcription noise.

To perform robust evaluation while maintaining reproducibility, LSIE-MLF uses Azure OpenAI Chat Completion models configured in deterministic mode. Temperature is fixed to zero, nucleus sampling is disabled, and structured JSON outputs are enforced through schema constraints.

The `is_match` boolean from the semantic evaluation output is consumed by the reward pipeline (§7B) as a binary gate $G_t \in \{0, 1\}$. When `is_match` is True, the encounter's P90 `AU12` intensity flows into the Thompson Sampling posterior update; when False, the reward is zeroed regardless of smile intensity. The `confidence_score` field is persisted to the Persistent Store for offline analysis but does not modulate the reward signal. This is an intentional design choice: continuous confidence gating would allow partially confident matches to drip reward into the posterior, accumulating false positive contamination over many encounters.

8.1 Inference parameter constraints

Parameter	Value	Justification
`temperature`	`0.0`	Collapses the probability distribution to deterministic token selection.
`top_p`	`1.0`	Disables nucleus sampling ensuring deterministic token selection.
`max_tokens`	`500`	Ensures adequate space for reasoning plus structured JSON output.
`response_format`	`JSONSchemaStructuredOutputs`	Forces the model to return machine-parseable JSON rather than conversational text.
`seed`	`42`	Activates Azure OpenAI deterministic inference mode.

8.2 Expected output schema

```
{
  "type": "object",
  "required": [
    "reasoning",
    "is_match",
    "confidence_score"
  ],
  "properties": {
    "reasoning": {
      "type": "string"
    },
    "is_match": {
      "type": "boolean"
    },
    "confidence_score": {
      "type": "number",
      "minimum": 0,
      "maximum": 1
    }
  },
  "additionalProperties": false
}
```

8.3 System prompt (canonical)

```
You are a highly precise linguistic evaluation assistant.
Your objective is to determine whether a transcribed 'Actual Utterance' is semantically equivalent
↪  to an 'Expected Greeting Rule'.

Inputs:
1. Expected Greeting Rule
2. Actual Utterance

Evaluation constraints:
- Minor transcription errors and filler words are acceptable if the semantic intent remains
↪  unchanged.
- Major deviations in subject matter must be rejected.

Steps:
1. Produce a reasoning explanation evaluating semantic similarity.
2. Output boolean is_match.
3. Output confidence_score between 0.00 and 1.00.

Return the result strictly as valid JSON without markdown formatting.
```

9. Docker compose runtime topology

The LSIE-MLF runtime environment consists of multiple containerized services orchestrated through Docker Compose. Containers share a common bridge network and specific named volumes enabling high-throughput IPC communication and persistent storage where required.

9.1 Container topology

Container	Role	Image	Configuration
`api`	API Server providing FastAPI REST endpoints.	`python:3.11-slim`	Port: 8000:8000 Net: appnetwork GPU: None Restart: `unless-stopped`
`worker`	ML Worker executing Celery inference tasks.	`nvidia/cuda:12.2.2-cudnn8-runtime-ubuntu22.04`	Port: None Net: appnetwork GPU: All (capabilities: [gpu], driver: nvidia) Restart: `on-failure:5`
`redis`	Message Broker used by Celery.	`redis:7-alpine`	Port: 6379 (internal only) Net: appnetwork GPU: None Restart: `unless-stopped`
`postgres`	Persistent Store for experiment and metric data.	`postgres:16-alpine`	Port: 5432 (internal only) Net: appnetwork GPU: None Restart: `unless-stopped`
`stream_scrcpy`	Capture Container extracting raw USB audio/video streams.	`ubuntu:24.04+scrcpyv3.3.4`	Port: None Net: appnetwork GPU: None Restart: `on-failure:3`
`orchestrator`	Module C orchestrator assembling 30-second segments, processing AU12 frames, and dispatching Celery tasks.	`nvidia/cuda:12.2.2-cudnn8-runtime-ubuntu22.04`	Port: None Net: appnetwork GPU: All (capabilities: [gpu], driver: nvidia) Restart: `on-failure:5`

9.2 Volume mounts

Volume	Mount path	Containers	Purpose
`ipc-share`	`/tmp/ipc/`	stream_scrcpy, worker, orchestrator	Shared volume containing IPC named pipes.
`data-raw`	`/data/raw/`	worker, orchestrator	Transient debug media buffers.
`data-interim`	`/data/interim/`	worker, orchestrator	Intermediate processing artifacts.
`data-processed`	`/data/processed/`	worker, orchestrator, api	Validated inference outputs and fallback storage.
`pg-data`	`/var/lib/postgresql/data/`	postgres	Persistent PostgreSQL database storage.

9.3 GPU allocation

The ML Worker and orchestrator containers receive GPU access through Docker's `deploy.resources` reservation syntax and require CUDA 12 with `cuDNN` 8 for `CTranslate2` compatibility. `cuDNN` 8 preserves Pascal SM 6.1 dp4a compatibility required by the target hardware.

```yaml
deploy:
  resources:
    reservations:
      devices:
        - driver: nvidia
          count: all
          capabilities: [gpu]
```

9.4 Device passthrough

USB passthrough allows the capture container to communicate directly with the Android device through the host kernel device subsystem.

```yaml
devices:
  - /dev/bus/usb:/dev/bus/usb
```

9.5 Network configuration

All services run within the same Docker bridge network allowing container name DNS resolution. Only the API service exposes ports externally.

9.6 Service dependency order

1. redis — starts first; health check redis-cli ping.
2. postgres — starts after redis; health check pg_isready.
3. stream_scrcpy — starts after redis to initialize IPC pipe.
4. worker — starts after redis, postgres, and stream_scrcpy.
5. orchestrator — starts alongside worker after redis, postgres, and stream_scrcpy.
6. api — starts after worker and postgres.

10. Dependency and runtime matrix

10.1 System requirements

Component	Required ver	Notes
Python	`3.11.x`	All containers MUST run Python 3.11. Python 3.12+ is untested with CTranslate2.
NVIDIA CUDA	`>=12.0`	Required for faster-whisper and CTranslate2 runtime compatibility.
cuDNN	`>=8.0`	Bundled within nvidia/cuda:12.2.2-cudnn8-runtime-ubuntu22.04; cuDNN 8 preserves Pascal SM 6.1 dp4a compatibility required by the hardening baseline.
NVIDIA Container Toolkit	`latest`	Required for Docker GPU passthrough.
GPU VRAM	`>=6GB`	Minimum VRAM for faster-whisper large-v3 INT8 plus MediaPipe runtime.
Docker Compose	`v2.x`	Required for deploy.resources GPU reservation syntax.
scrcpy	`3.3.4`	Required for raw Android USB audio/video extraction; v3.3.4 provides the –no-playback flag and dual-instance –port range support used by the capture entrypoint.

10.2 Python package pinning

Package	Version	Container(s)	Purpose
`faster-whisper`	`1.2.1`	worker	GPU accelerated speech transcription
`mediapipe`	`0.10.9`	worker	Face Mesh 478 landmark inference
`praat-parselmouth`	`0.4.4`	worker	Praat acoustic feature extraction
`spacy`	`3.7.x`	worker	NLP preprocessing pipeline
`psycopg2-binary`	`2.9.x`	worker, api	PostgreSQL driver
`pandas`	`2.2.x`	worker	Time-series data processing
`scipy`	`1.13.*`	worker	Statistical computation
`celery`	`5.4.x`	worker, api	Distributed task queue
`redis`	`5.x`	worker, api	Celery broker transport
`fastapi`	`0.110.x`	api	REST API framework
`uvicorn`	`0.29.x`	api	ASGI server runtime
`pydantic`	`2.x`	worker, api	Schema validation
`numpy`	`1.26.x`	worker	Vectorized AU12 calculations
`pycryptodome`	`>=3.20.0`	worker	AES-256-GCM encryption
`patchright`	`1.58.2`	worker	Stealth browser automation
`TikTokLive`	`5.0.x`	worker	TikTok WebSocket ingestion

Package	Version	Container(s)	Purpose
`streamlit`	`1.*`	worker	Analytics dashboard

11. Variable extraction matrix

Each analytical variable produced within LSIE-MLF is traceable to its source module, data type, storage location, and extraction methodology.

Variable Profile	Extraction & Routing
Raw Video Frames `bytestream(H.264)` Module A → Transient (kernel buffer)	`scrcpy` USB mirroring; H.264 stream via `/dev/bus/usb` passthrough.
Raw Audio PCM `bytestream(s16le)` **Unit:** 48 kHz mono Module A → Transient IPC pipe buffer	`scrcpy` raw audio capture; `s16le` 48 kHz mono to `IPC` Pipe.
Live Event Payloads `dict` Module B → Transient deque buffer	`TikTokLive` `WebSocket` ingestion; parsed protobuf event dictionaries.
Action Combo Trigger `bool` **Unit:** True/False Module B → Persistent Store events table	Composite temporal event correlation across concurrent chat actions.
UTC Drift Offset `float` **Unit:** seconds Module C → Transient variable	`ADB` epoch polling every 30s; `drift_offset` = host UTC − Android epoch.
Resampled Audio Chunks `bytestream(s16le)` **Unit:** 16 kHz mono Module C → Transient stdout pipe	`FFmpeg` resampling subprocess; 48 kHz → 16 kHz mono `s16le` on `pipe:1`.
3D Facial Landmarks `np.ndarray(478,3)` **Unit:** normalized coordinates Module D → Transient memory	`MediaPipe` `FaceMesh`; 478-vertex 3D landmark array per frame.
AU12 Intensity Score `float` **Unit:** 0.0–5.0 (compute_intensity) or 0.0–1.0 (compute_bounded_intensity for Thompson Sampling reward pipeline) Module D → Persistent Store metrics table	`AU12Normalizer` (§7A); `IOD`-normalized lip-corner distance with neutral baseline subtraction. Exposed as 0.0–5.0 (legacy) and 0.0–1.0 (bounded, for reward pipeline).

Variable Profile	Extraction & Routing
Vocal Pitch F0 `float` **Unit:** 75–600 Hz Module D → Persistent Store metrics table	`Praat` via `parselmouth`; fundamental frequency extraction from speech audio.
Jitter `float` **Unit:** 0–1 ratio Module D → Persistent Store metrics table	`Praat` via `parselmouth`; cycle-to-cycle pitch perturbation ratio.
Shimmer `float` **Unit:** 0–1 ratio Module D → Persistent Store metrics table	`Praat` via `parselmouth`; cycle-to-cycle amplitude perturbation ratio.
ASR Transcription `string` **Unit:** UTF-8 text Module D → Persistent Store transcripts table	`faster-whisper` with `CTranslate2` INT8 backend; segment-level speech-to-text.
Semantic Match `JSONobject` **Unit:** confidence 0–1 Module D → Persistent Store evaluation table	Azure `OpenAI` deterministic evaluation (§8); `is_match`, `confidence_score`, `reasoning`.
Evaluation Variance `float` **Unit:** 0–infinity Module E → Persistent Store experiments table	Thompson Sampling posterior statistics; variance of Beta distribution per arm.
External Context Metadata `JSONdict` Module F → Persistent Store context table	`patchright` headless browser scraping; creator and trending topic metadata.
gated_reward `float` **Unit:** [0.0, 1.0] Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); `r_t` = `P90` × `semantic_gate`.
p90_intensity `float` **Unit:** [0.0, 1.0] Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); 90th percentile of bounded `AU12` in stimulus window.
semantic_gate `int` **Unit:** {0, 1} Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); binary gate from `is_match`: 1 if True, 0 otherwise.

Variable Profile	Extraction & Routing
n_frames_in_window `int` **Unit:** [0, +infinity) Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); count of `AU12` observations in stimulus window.
baseline_b_neutral `float|null` **Unit:** [0.0, +infinity) or null Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); mean bounded `AU12` in pre-stimulus window, or null.
stimulus_time `float` **Unit:** UTC epoch seconds Module C → Transient variable	Drift-corrected UTC epoch of stimulus injection; defines baseline and measurement windows for `compute_reward()` (§7B).

12. Error handling and recovery specification

LSIE-MLF defines four standardized failure categories. Each module must implement deterministic recovery behavior for each category.

12.1 Network disconnection

Module	Behavior
Module A	N/A
Module B	Exponential `WebSocket` reconnection up to 10 retries
Module C	N/A
Module D	Retry Azure `OpenAI` request once then record null
Module E	Buffer database writes and retry every 5s
Module F	Return empty scrape result

12.2 Hardware device loss

Module	Behavior
Module A	USB reconnection polling every 2s for 60s then container restart
Module B	N/A

Module	Behavior
Module C	Freeze drift offset after 3 consecutive `ADB` poll failures (`DRIFT_FREEZE_AFTER_FAILURES` = 3). While frozen, return the cached offset without additional `ADB` calls; reset drift to zero after 300 seconds / 5 minutes of frozen state (`DRIFT_RESET_TIMEOUT` = 300).
Module D	Terminate worker container if GPU unavailable
Module E	N/A
Module F	N/A

12.3 Worker process crash

Module	Behavior
Module A	Pipe overflow handled via `EAGAIN` non-blocking writes
Module B	Reconnect `WebSocket` on restart
Module C	Restart `FFmpeg` within 1 second when `AudioResampler` detects a dead subprocess or `EOF`. If the video capture thread dies after an `IPC` / `PyAV` pipe break, the run loop recreates and restarts `VideoCapture` so the upstream `scrcpy` path can emit a fresh `MKV` header.
Module D	Docker restart policy on-failure:5
Module E	Log `Celery` task failure and continue
Module F	Retry scraping task up to two times

12.4 Queue overload

Module	Behavior
Module A	Kernel pipe overflow discards data silently
Module B	Deque ring buffer evicts oldest entries
Module C	Deque maxlen eviction policy
Module D	`Celery` queue buffers tasks in `Redis`
Module E	Overflow records written to CSV fallback storage
Module F	Limit concurrent scraping tasks

13. Autonomous implementation audit checklist

Before deployment the system must satisfy every verification item defined below to ensure deterministic implementation.

#	Audit item	Verification criterion
1	**Directory structure**	Repository directories match the hierarchy defined in Section 3.
2	**Docker topology**	Six containers are defined with correct images, networks, volumes, and restart policies.
3	**IPC lifecycle**	Named pipe lifecycle implemented according to Section 4.A.2.
4	**Audio pipeline**	`FFmpeg` resampling from 48 kHz to 16 kHz executed correctly.
5	**Drift correction**	`ADB` polling every 30 seconds with `drift_offset` applied.
6	**AU12 implementation**	Landmark indices and epsilon guard implemented correctly.
7	**LLM determinism**	Azure `OpenAI` configured with temperature 0 and structured outputs.
8	**Ephemeral Vault**	AES-256-GCM encryption and 24-hour secure deletion enforced.
9	**Schema validation**	`InferenceHandoffPayload` validated via `Pydantic`.
10	**Module contracts**	Inputs, outputs, dependencies, side effects, and failures implemented exactly as specified.
11	**Error handling**	All four failure categories implemented across modules.
12	**Dependency versions**	Pinned runtime versions match the dependency matrix.
13	**Variable traceability**	All variables defined in the extraction matrix are produced by the correct module.
14	**Data classification**	Transient, debug, and permanent storage tiers enforced.
15	**Canonical terminology**	Only canonical component names are used throughout code and configuration.
16	**Reward pipeline**	Thompson Sampling reward pipeline implements the mathematical recipe from §7B: P90 aggregation, binary semantic gate, fractional Beta-Bernoulli posterior updates with Beta(1,1) prior, and stimulus window boundaries [t+0.5s, t+5.0s].

— END OF SPECIFICATION —